

Data Structures

CSC 301

Lecture Notes

Chapter 3: Basic Data Structures: Arrays, Records, and Strings

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

3	Basic Data Structures: Arrays, Records, and Strings	2
3.1	Introduction	2
3.1.1	Arrays	2
3.1.2	Records	3
3.1.3	Strings	3
3.1.4	String Processing	3
3.1.5	Conclusion	4

Chapter 3

Basic Data Structures: Arrays, Records, and Strings

[TERMINOLOGY] [/TERMINOLOGY]

[CHAPTER]

3.1 Introduction

This chapter introduces three fundamental data structures that serve as the building blocks for more complex structures: **Arrays**, **Records**, and **Strings**. Each of these structures provides a distinct way to organize and access data in memory, and each is closely tied to the underlying memory representation of the data it holds. Understanding these structures is essential before proceeding to the linked structures, stacks, queues, and trees discussed in later chapters.

3.1.1 Arrays

An **array** is a collection of elements stored in contiguous memory locations, where each element can be accessed directly by its index. The index is typically an integer that ranges from zero to one less than the total number of elements. Because the elements are stored contiguously, the address of any element can be computed in constant time using the base address of the array and the index of the desired element.

For example, consider an array of five integers. The element at index zero occupies the first memory location, the element at index one occupies the next, and so on. This layout allows random access to any element in constant time, denoted as $O(1)$. However, the size of a static array is fixed at the time of creation, and inserting or removing elements in the middle of the array requires shifting subsequent elements, which takes linear time.

Arrays can be allocated on the stack or on the heap. Stack allocation is suitable for arrays whose size is known at compile time and whose lifetime is limited to the scope in which

they are declared. Heap allocation is used when the size is not known until run time or when the array must outlive the function that created it.

3.1.2 Records

A **record** is a composite data structure that groups together a fixed number of fields, each of which may have a different data type. Unlike an array, where all elements share the same type, a record allows heterogeneous data to be stored together under a single name. Each field within a record is identified by a name or a tag, and the fields are accessed individually rather than by a numeric index.

For instance, a record representing a student might contain a field for the student's name, a field for the student's age, and a field for the student's grade point average. The fields are stored in a fixed order, and the memory layout of the record is determined by the order and types of its fields. Memory alignment and padding may be applied to ensure that each field begins at an address that is a multiple of its size, which can affect the total size of the record.

Records are useful for modeling real-world entities that have multiple attributes. They are sometimes referred to as structs in certain programming languages, but the underlying concept remains the same.

3.1.3 Strings

A **string** is a sequence of characters used to represent text. In most programming languages, strings are stored as arrays of characters, with an additional convention to mark the end of the sequence. In some languages, a special null character is appended to the end of the character array to indicate termination. In other languages, the length of the string is stored separately as part of the string object.

Strings can be either mutable or immutable. In a mutable string, the characters can be modified after the string is created. In an immutable string, any operation that appears to modify the string actually creates a new string object. The choice between mutability and immutability affects both the performance and the safety of string operations.

3.1.4 String Processing

String processing refers to the set of operations and algorithms used to manipulate and analyze text data. Common operations include concatenation, which joins two strings into one; substring extraction, which retrieves a contiguous portion of a string; and searching, which locates a pattern within a string. More advanced operations include pattern matching using regular expressions, sorting of collections of strings, and transformation of text according to specified rules.

Many programming languages provide built-in functions or library routines for these operations. For example, a concatenation function might take two strings as input and return a new string that contains the characters of the first followed by the characters of

the second. A search function might take a string and a pattern as input and return the index at which the pattern first appears in the string.

3.1.5 Conclusion

Arrays, records, and strings are foundational data structures that provide efficient ways to store and access collections of data. Arrays offer constant-time indexed access to elements of the same type. Records allow heterogeneous data to be grouped under a single name. Strings provide a specialized representation for text that supports a rich set of processing operations. Together, these structures form the basis for the more advanced data structures that are explored in subsequent chapters.

Tutorial Questions

1. Define an array and explain how the index of an element is used to compute its memory address.
2. Compare stack allocation and heap allocation in the context of array storage.
3. Describe the difference between an array and a record in terms of element types and access methods.
4. Explain how memory alignment and padding can affect the size of a record in memory.
5. What is the difference between a mutable string and an immutable string?
6. List four common operations in string processing and briefly describe each.
7. Given an array of integers containing the values 7, 2, 9, 4, and 1, what value is stored at index 3 if indexing begins at zero?

[/CHAPTER]